Final Delivery – Model Serving and End-to-End MLOps Pipeline

Unidade Curricular: Engenharia de Modelos Inteligentes
Docente: António Raimundo
Ano Letivo: 2025/2026

Cátia Brás, 120093
David Volovei, 120051
Duarte Cabrita, 120058
Eduardo Martins, 120063

1. Introduction 3
2. Problem Definition and Dataset 3
3. System Architecture 3
4. Data Versioning with DVC 4
5. Experiment Tracking with MLflow 5
6. Configuration Management with Hydra 6
7. Data Ingestion and API Simulation 6
8. Pipeline Orchestration with Apache Airflow 7
9. Data Stream DAG 7
10. Reactive Training DAG 8
11. Model Training and Evaluation 9
12. Model Serving with FastAPI 9
13. Explainability with SHAP 11
14. Frontend Application 11
15. Containerization with Docker Compose 12
16. End-to-End Pipeline Demonstration 13
17. Conclusion 14

1. Introduction

The present project aims to develop a complete end-to-end Machine Learning Operations (MLOps) pipeline for predicting housing prices in King County, USA. Rather than focusing solely on predictive accuracy, the primary objective is to design a robust, reproducible, and scalable system that reflects modern machine learning engineering practices. According to the project guidelines , emphasis is placed on the full lifecycle of machine learning systems, including data versioning, experiment tracking, configuration management, pipeline orchestration, and model serving.

This work builds upon the initial foundation established in Tutorial #1, where data versioning and experiment tracking were introduced, and extends it into a fully automated and production-oriented pipeline. The final solution integrates multiple tools and frameworks, including DVC, MLflow, Hydra, Apache Airflow, FastAPI, and Streamlit, ensuring a modular and extensible architecture.

2. Problem Definition and Dataset

The problem addressed in this project consists of a supervised regression task, where the goal is to predict the selling price of residential properties based on a set of physical and geographical features. The dataset used is the King County Housing Dataset, which contains information such as the number of bedrooms, bathrooms, living area, lot size, construction year, and location-specific attributes.

As described in the first tutorial , the analytical objective is to model the relationship between these features and the final sale price of each property. This dataset provides a realistic and well-structured scenario for testing machine learning models, while also allowing the exploration of MLOps concepts in a practical context.

3. System Architecture

The system was designed as a modular MLOps pipeline composed of several interconnected components. The workflow begins with data ingestion, followed by preprocessing, model training, evaluation, and registration, and finally model serving through an API connected to a frontend application. Each component is designed to operate independently while maintaining strong integration with the rest of the system.

The overall architecture ensures that new data can be continuously incorporated into the system, triggering automated retraining and updating of the deployed model. This design reflects a real-world production environment, where machine learning systems must adapt dynamically to new information while maintaining consistency and reproducibility.

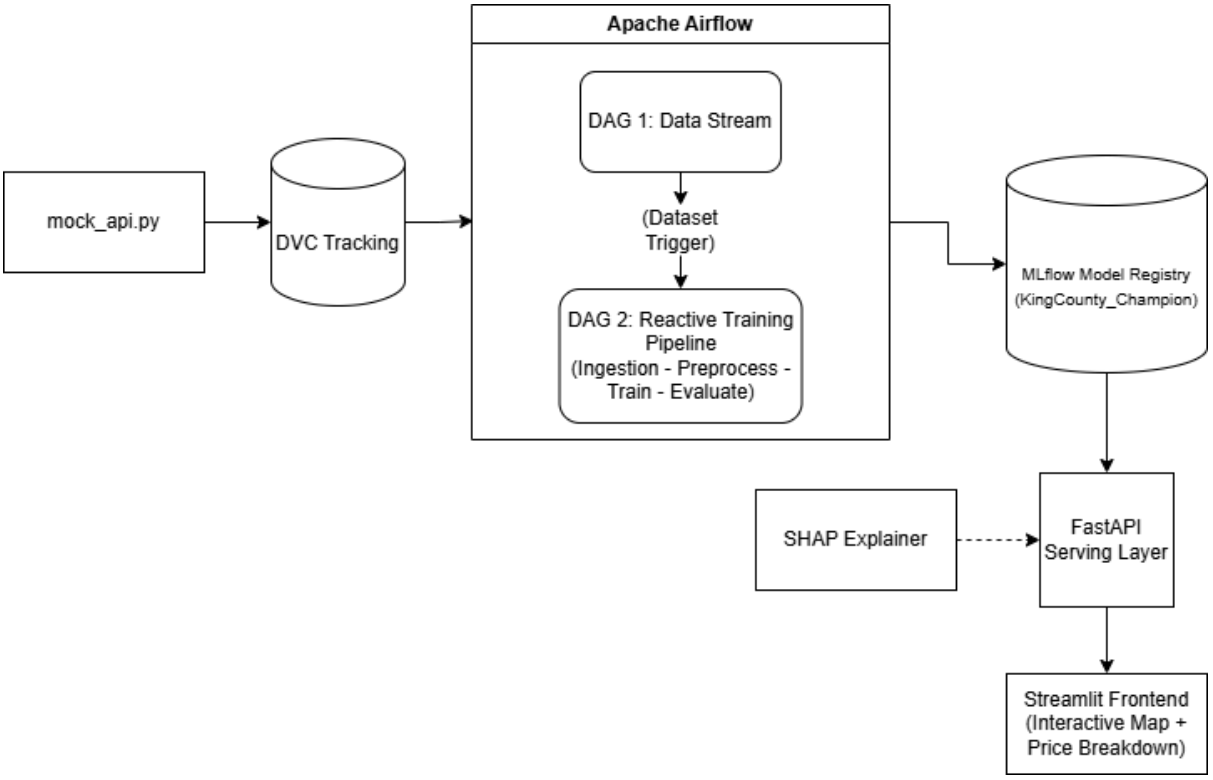

Figure 1 – Architecture diagram

The entire infrastructure is containerized using Docker Compose, which orchestrates seven services: PostgreSQL (Airflow metadata), MLflow Tracking Server, three Airflow components (scheduler, webserver, DAG processor), FastAPI, and Streamlit. This ensures that the system can be launched fully with a single “docker compose up –build”, guaranteeing reproducibility across environments.

4. Data Versioning with DVC

Data versioning is implemented using DVC (Data Version Control), which enables tracking of dataset versions through content-based hashing. The dataset is divided into two main components: a baseline dataset, which is continuously updated, and a pool of future data batches used to simulate incoming data.

Each update to the dataset is tracked by DVC, which generates an MD5 hash representing the exact state of the data at a given moment. This mechanism ensures strict data lineage, allowing every experiment to be associated with a specific dataset

version. As a result, it is always possible to reproduce previous experiments and verify their results, which is a fundamental requirement in MLOps systems.

5. Experiment Tracking with MLflow

MLflow is used as the experiment tracking platform, providing a centralized interface for logging and comparing model runs. During each execution, relevant information such as hyperparameters, evaluation metrics, and metadata is recorded. In particular, the system logs performance metrics including RMSE, MAE, and R^2, which are used to evaluate model quality.

Additionally, the integration between MLflow and DVC ensures that each run includes the dataset hash corresponding to the data used during training. This connection establishes a clear link between data and results, enabling full traceability. As shown in the MLflow screenshots, multiple runs are tracked and can be compared, while detailed information about each model, including parameters and performance, is readily accessible.

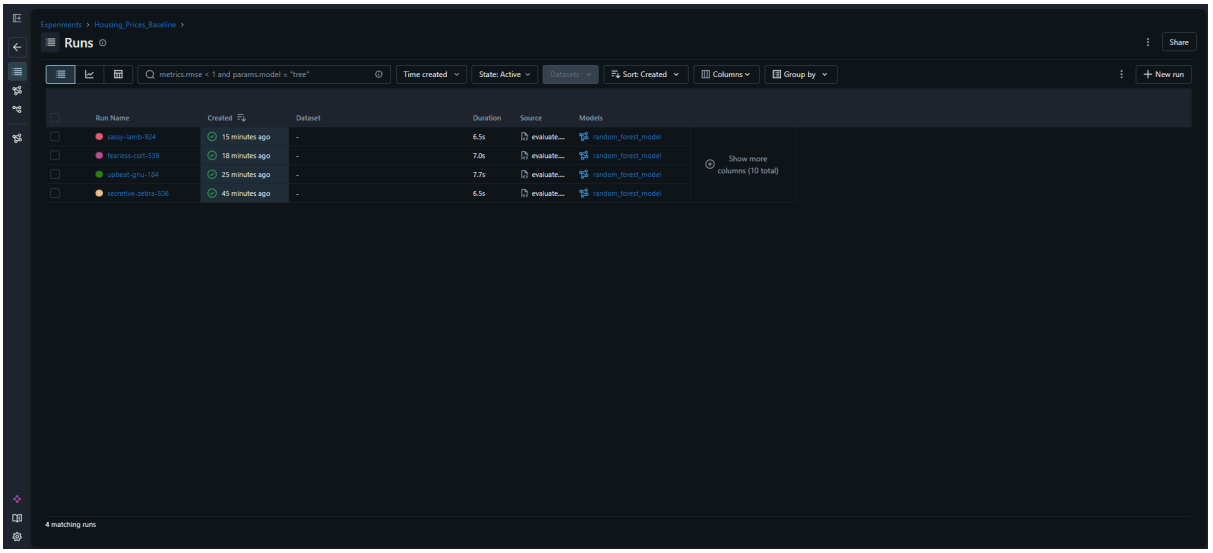

Figure 2 – MLflow Runs Overview

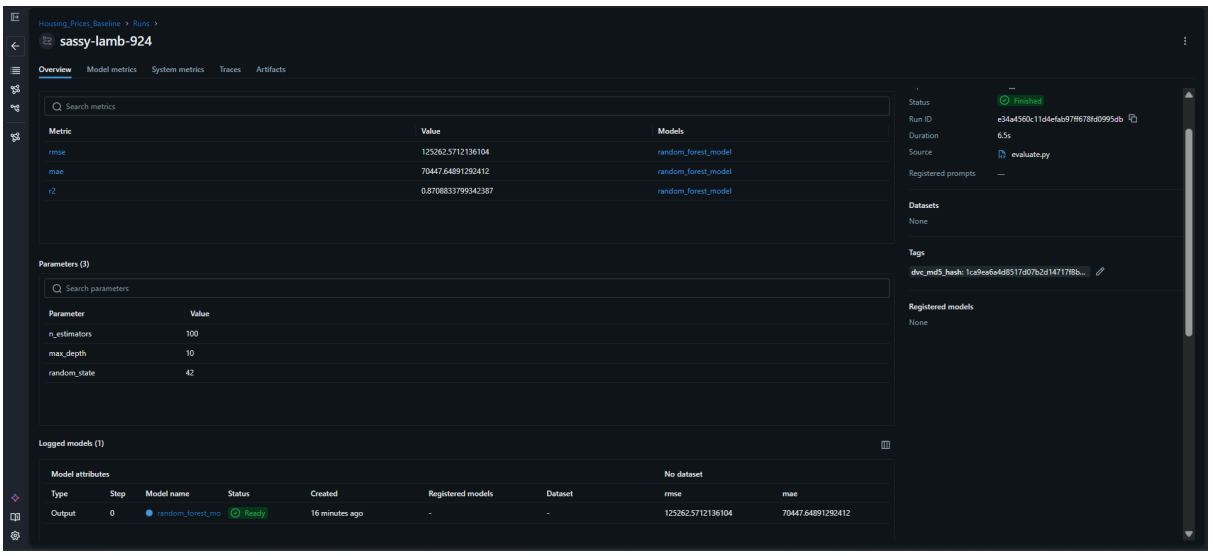

Figure 3 – MLflow Model Details and Metrics

6. Configuration Management with Hydra

Hydra is used to manage all configuration parameters dynamically, replacing hardcoded values with structured configuration files. These configurations include file paths, model hyperparameters, and API-related settings, all defined in hierarchical YAML files.

This approach significantly improves the flexibility and maintainability of the system, as changes can be made without modifying the source code. Furthermore, Hydra enables easy experimentation by allowing different configurations to be executed simply by changing input parameters. This reinforces reproducibility and aligns with best practices in machine learning engineering.

7. Data Ingestion and API Simulation

To simulate a real-world production environment, a mock API was implemented to emulate the continuous arrival of new data. Instead of relying on a static dataset, the system reads from a pool of future data and releases new records incrementally.

Each execution of the ingestion process appends new data to the baseline dataset, which is then tracked by DVC. This approach allows the system to operate in a dynamic setting, where new data continuously influences the model. By simulating an API, the project introduces a realistic data ingestion mechanism that prepares the pipeline for deployment in real-world scenarios.

8. Pipeline Orchestration with Apache Airflow

Pipeline orchestration is handled using Apache Airflow, which organizes the workflow into Directed Acyclic Graphs (DAGs). The system includes two main DAGs: one responsible for simulating data ingestion and updating the dataset, and another responsible for executing the machine learning pipeline.

As shown in the Airflow overview screenshot, both DAGs are active and successfully executed, demonstrating the correct configuration of the orchestration layer. The data stream DAG operates on a scheduled basis, while the reactive training DAG is triggered automatically when changes in the dataset are detected.

The execution logs further confirm that both DAGs run successfully, with all tasks completed without errors. This demonstrates that the pipeline is fully automated and capable of handling data updates in a reliable manner.

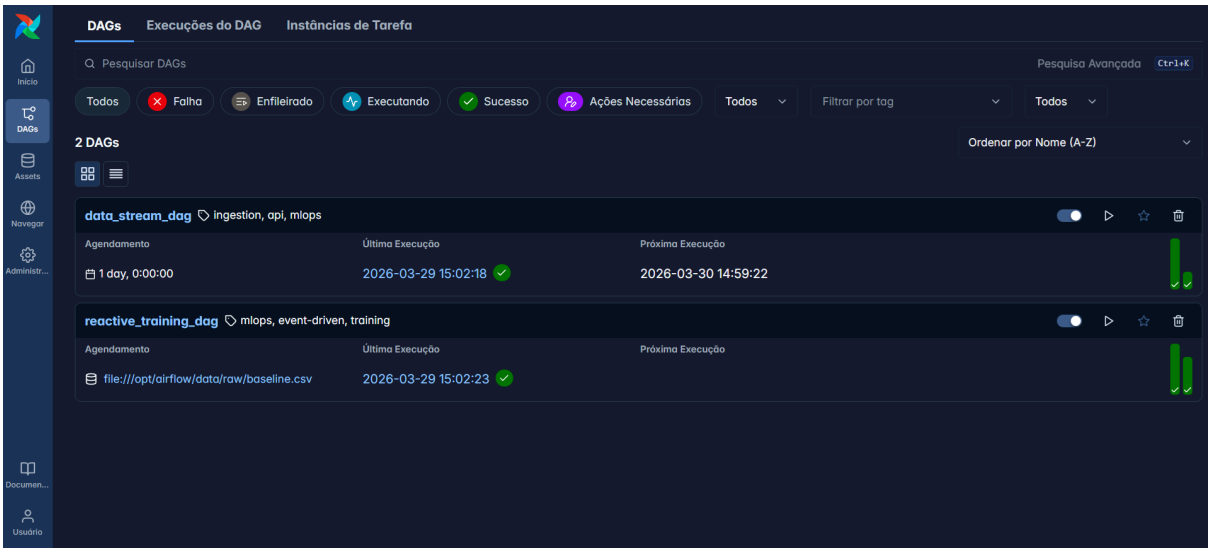

Figure 4 - Airflow DAGs Overview

9. Data Stream DAG

The data stream DAG is responsible for simulating API calls and updating the dataset. It consists of two main tasks: fetching new data from the mock API and updating the dataset tracked by DVC.

The task-level view shows that both tasks are executed successfully, confirming that the ingestion process is functioning correctly. This DAG ensures that new data is continuously integrated into the system, serving as the entry point for the entire pipeline.

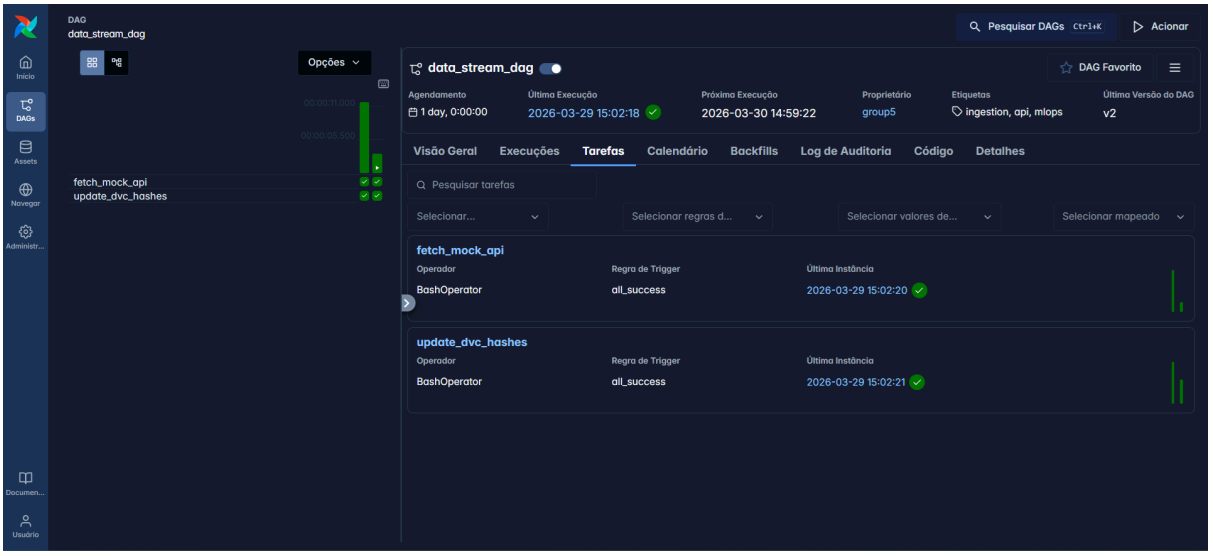

Figure 5 – Data Stream DAG Tasks Execution

10. Reactive Training DAG

The reactive training DAG is triggered automatically when the dataset is updated, representing an event-driven pipeline. This behavior is confirmed in the execution logs, where the DAG is marked as “asset_triggered”, indicating that it was executed in response to data changes.

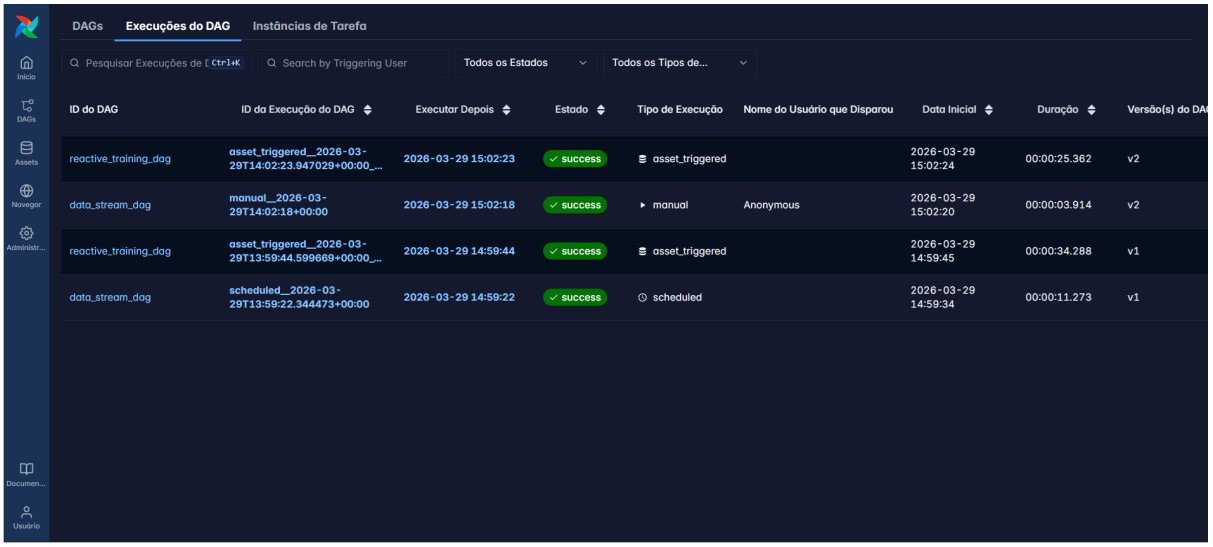

Figure 6 – Reactive DAG Trigger Execution

The DAG executes a sequence of tasks corresponding to data ingestion validation, preprocessing, model training, and evaluation. The task visualization confirms that each step is completed successfully and in the correct order. This ensures that the entire machine learning workflow is executed automatically whenever new data becomes available.

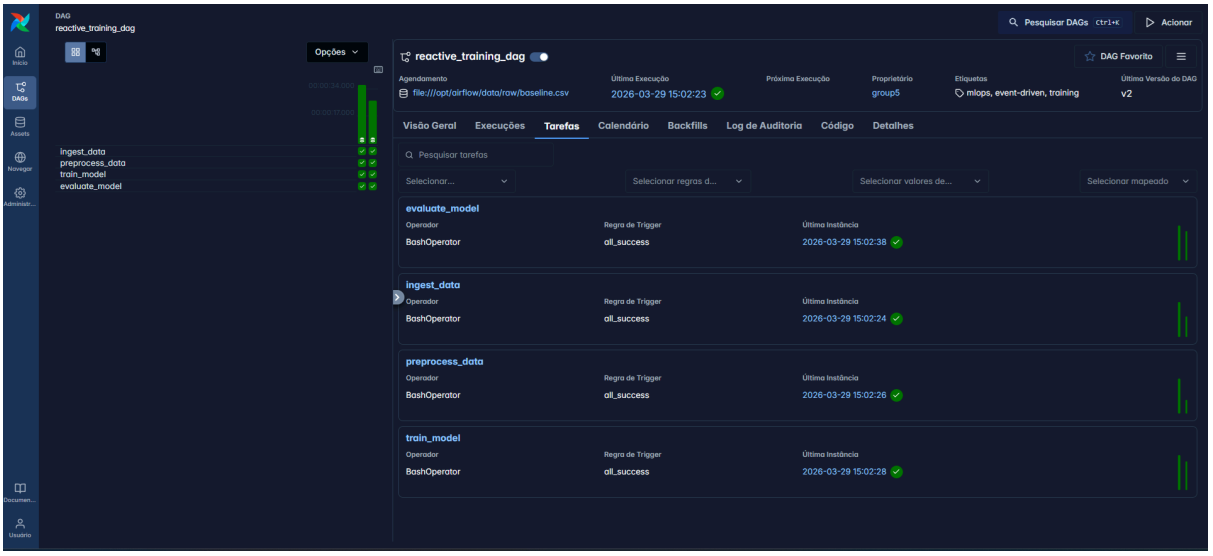

Figure 7 – Reactive Training DAG Tasks Execution

11. Model Training and Evaluation

Model training is performed using three different algorithms: Random Forest, Linear Regression, and XGBoost. This multi-model approach allows for a comparative evaluation, ensuring that the best-performing model is selected.

During the evaluation phase, each model is assessed using standard regression metrics, and the model with the lowest RMSE is selected as the champion.

The results of the tournament indicate that the XGBoost model consistently achieves the lowest RMSE (approximately 106,000), followed by Random Forest (approximately 125,000) and Linear Regression (approximately 206,000). Only the champion model is registered in the MLflow Model Registry under the name "KingCounty_Champion", making it immediately available for serving.

The tournament design ensures that the system always serves the best available model. If a configuration change (e.g., increasing the number of estimators) causes a different model to outperform the current champion, the pipeline automatically promotes that new model to the “champion” status and serves it. All three models’ metrics (RMSE, MAE, R^2) are logged in the same MLflow run, enabling side-by-side comparison in the MLflow UI.

12. Model Serving with FastAPI

A FastAPI-based service is implemented to expose the trained model through RESTful endpoints. Upon startup, the service dynamically loads the latest champion

model from the MLflow Model Registry using a retry mechanism that handles scenarios where the model may be temporarily unavailable during retraining.

The API enforces strict input validation using Pydantic models. The HousingInput schema defines data type constraints and domain-specific bounds for each feature, for example, latitude must fall within King County's range (47.0 - 48), and the construction grade must be between 1 and 13. Invalid requests are rejected before reaching the model, preventing erroneous predictions.

In addition to the predicted price, the "/predict" endpoint returns:

- The name of the champion algorithm (e.g. "XGBoost")
- The champion's RMSE, used by the frontend to display confidence intervals
- Per-feature SHAP contributions, enabling the explainability visualization

A health check endpoint ("GET /") is also available, returning the current model status.

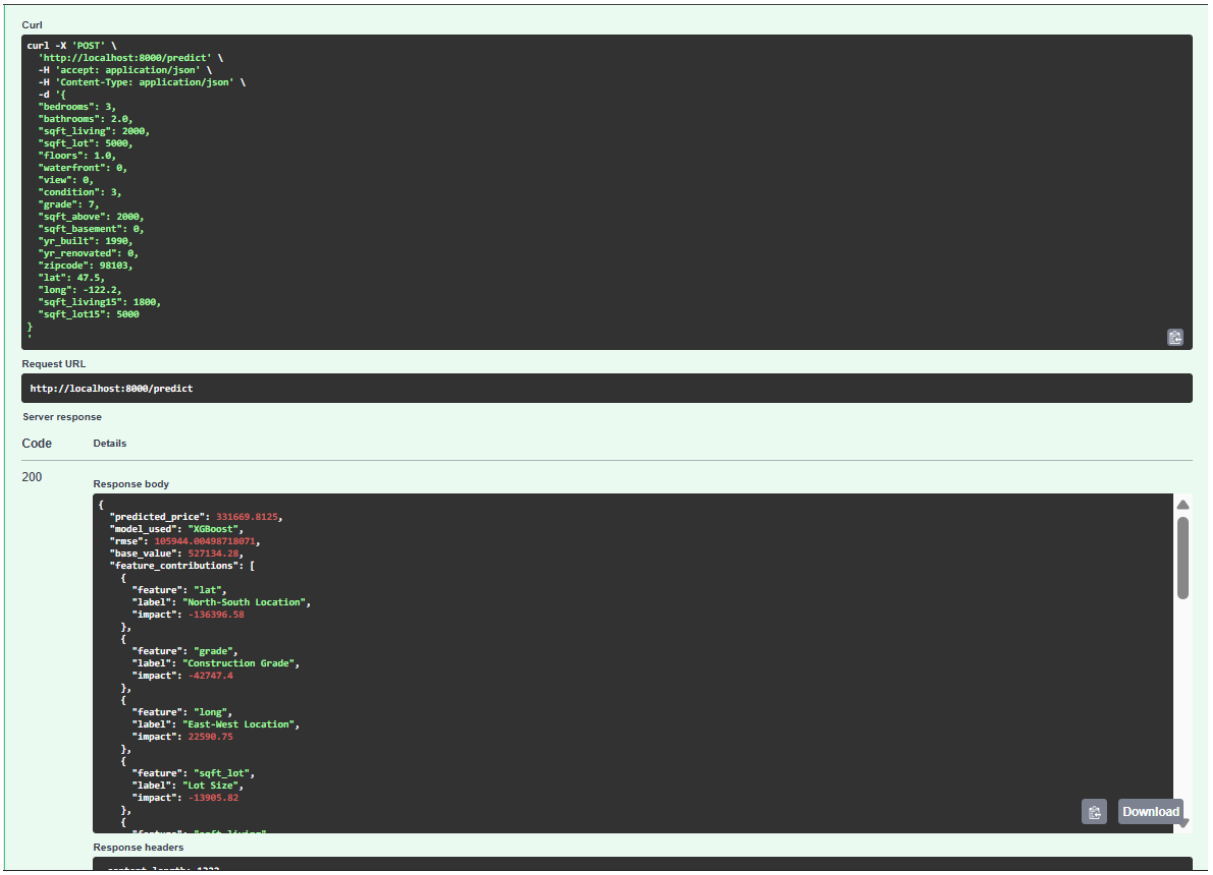

Figure 8 – FastAPI UI with the /predict endpoint

13. Explainability with SHAP

To enhance the transparency of predictions, the system integrates SHAP (SHapley Additive exPlanations), a method rooted in cooperative game theory that quantifies each feature's individual contribution to a prediction.

When a user submits a prediction request, the API computes SHAP values using a PermutationExplainer. This approach treats the model as a black-box callable (model.predict), making it compatible with all three tournament contenders (Random Forest, Linear Regression, and XGBoost) without requiring model-specific code paths. The explainer uses a background sample of 100 training instances, loaded at startup, as the reference distribution against which feature contributions are measured.

For each prediction, the API returns a base_value, the model's average prediction across the background sample, along with a list of per-feature impacts sorted by absolute magnitude. These values satisfy the SHAP additivity property: base_value + Σ(feature contributions) ≈ predicted_price.

The frontend visualizes these contributions as a horizontal bar chart, with an expandable detailed breakdown in plain language (e.g., "Living Area: adds +$45,000 to the price"). This allows non-technical users to understand why a property is valued the way it is, not just what the predicted price is.

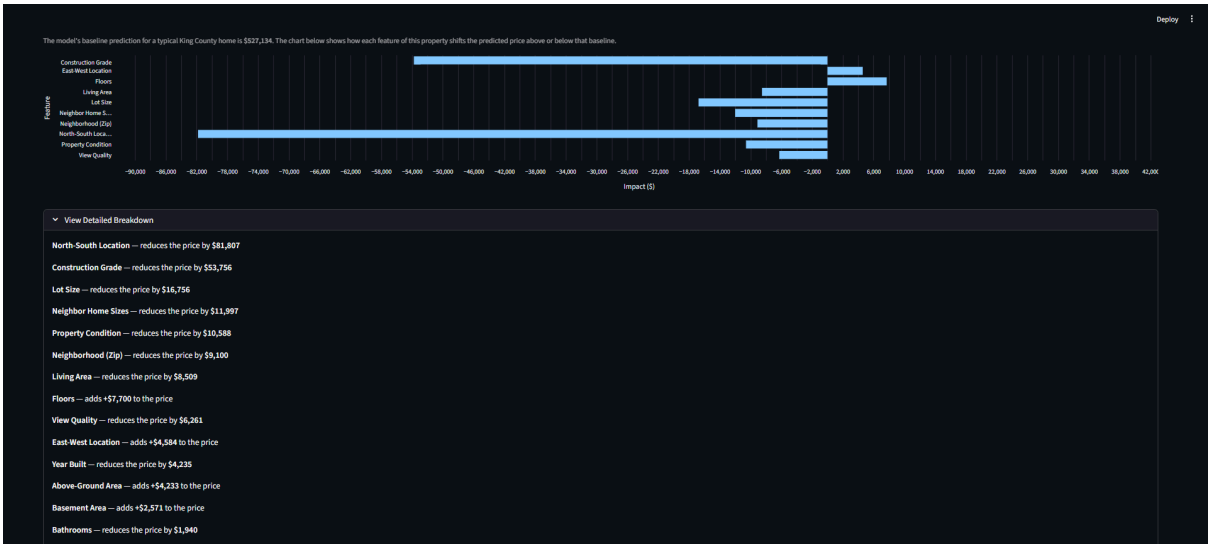

Figure 9 – Price Breakdown using SHAP on the UI

14. Frontend Application

The Streamlit application was developed to provide a user-friendly interface for interacting with the prediction system. The application is structured in three main sections:

- Location Selection: Users can choose a property location either by clicking on an interactive Folium map (which displays the King County boundary) or by entering exact latitude and longitude coordinates manually.
- Property Details: A structured input form allows users to specify property characteristics across three categories: Interior Basics (bedrooms, bathrooms, floors), Size & Area (living area, lot size, basement), and Condition & History (construction grade, condition score, year built). Advanced features such as waterfront access, view score, and neighbor property sizes are available in an expandable section.
- Valuation Dashboard: After submitting a prediction, the dashboard displays the estimated market value with an optional confidence interval toggle (computed as ±RMSE). Below the valuation, a SHAP-powered feature impact chart shows which property attributes are adding or reducing value relative to a typical King County home.

The frontend communicates with the FastAPI backend via HTTP POST requests, and all state is managed through Streamlit's session state to maintain a responsive user experience.

15. Containerization with Docker Compose

The entire infrastructure is containerized using Docker Compose, enabling the full system to be launched with a single command (docker compose up --build). The architecture consists of seven services:

Service	Role	Port
PostgreSQL	Airflow metadata database	5432
MLflow Server	Experiment tracking and model registry	5000
Airflow Webserver	DAG management UI	8081
Airflow Scheduler	DAG execution scheduling	-
Airflow DAG Processor	DAG file parsing	-
FastAPI	Model Serving and SHAP explainability	8000
Streamlit	User-friendly interface	8501

All services share a common Docker image built from a custom Dockerfile that extends the official Apache Airflow image with the project's Python dependencies. Volume mounts ensure that source code, configuration, data, and MLflow artifacts are shared across containers, eliminating the need for file duplication.
The Airflow services connect to MLflow via the internal Docker network (http://mlflow_server:5000), while the FastAPI service loads models directly from the shared MLflow artifact store. This design mirrors a production deployment topology where services communicate over internal networks rather than through the host.

16. End-to-End Pipeline Demonstration

To demonstrate the complete end-to-end workflow as specified in the project guidelines, the following sequence can be executed:

1. Configuration Update: Modify a hyperparameter in conf/config.yaml, for example, changing xgboost.n_estimators from 200 to 300.
2. Pipeline Trigger: Manually trigger the Data Stream DAG in the Airflow UI, or wait for the scheduled execution. This fetches new data and updates the DVC-tracked baseline.
3. Automatic Retraining: The Reactive Training DAG is triggered automatically by the dataset change. The pipeline re-executes ingestion, preprocessing, training (with the updated hyperparameters), and evaluation.
4. Model Verification: In the MLflow UI, the new run appears with the updated xgb_n_estimators=300 parameter logged. If XGBoost remains the champion, a new model version is registered in the Model Registry.
5. API Update: The FastAPI service automatically loads the latest registered model version on the next prediction request, serving the updated model without redeployment.

This sequence validates that a single configuration change propagates through the entire system (from Hydra to Airflow to MLflow to the serving layer) without any manual intervention beyond the initial trigger.

17. Conclusion

This project successfully implements a complete, production-ready MLOps pipeline for King County housing price prediction. The integration of DVC, MLflow, Hydra, Apache Airflow, FastAPI, and Streamlit results in a system that is robust, reproducible, and scalable.

Key achievements include the event-driven two-DAG architecture, which automatically retrains models in response to new data; the three-model tournament, which ensures the best-performing algorithm is always deployed; the SHAP-based explainability layer, which provides transparent and user-friendly predictions; and the full containerization via Docker Compose, which enables the entire infrastructure to be launched with a single command.

The system demonstrates the ability to handle dynamic data, automate the machine learning workflow, and serve interpretable predictions in real time. Potential future improvements include continuous hyperparameter optimization using tools such as Optuna, integration of model monitoring and drift detection, and deployment to cloud infrastructure for horizontal scaling.